

Worksheet Answers: The Transformer

CS 5090 · Software Engineering with Generative AI · Week 3 · answers with short explanations, for your own review. These are the kinds of questions technical interviews ask, so the explanations matter more than the letters.

Part A. Quick checks

1. What does the softmax of the query–key scores produce? **(b)** weights saying how much each position contributes to this position's new representation. Note there are two different softmaxes in a language model: this one is over *positions* inside attention; the one that produces the next-token distribution is over the *vocabulary*, at the output head.
2. Why masked (causal) self-attention? **(c)** a position cannot attend to positions after it, which matches how generation works: at inference the future does not exist yet.
3. Match each architecture to its use. encoder-only (BERT) = **(c)** encode an input and predict from the embedding; decoder-only (GPT) = **(a)** generating code left-to-right; encoder–decoder = **(b)** translation.
4. Do all heads share one set of Q/K/V projections? **False**. Each head has its own Q/K/V projections, which is the whole point: different heads learn different relationships. W_O is a learned recombination of the concatenated head outputs, not an average.
5. What is one Transformer block? **(b)** self-attention, then a feed-forward network, each wrapped in a residual connection and layer normalization.
6. What does the KV cache store? **(b)** the keys and values already computed for earlier tokens, so each new token attends to them without recomputing the whole prefix. That is why generating token 500 does not cost 500 times what token 1 cost.

Part B. Work it by hand

7. Causal attention mask for four tokens. The lower triangle is allowed, diagonal included: a token may attend to itself.

	t1	t2	t3	t4
t1	✓			
t2	✓	✓		
t3	✓	✓	✓	
t4	✓	✓	✓	✓

8. Attention weights for scores 2, 1, 0.

The exponentials are 7.4, 2.7 and 1, summing to 11.1, so the weights are **0.67, 0.24 and 0.09** and the token with score 2 dominates. The point to carry away: exponentiation widens the gaps, so a modest lead in raw score becomes a large lead in attention weight, and the weights always sum to 1.

Part C. Short answers

9. What did the Transformer remove, and why was training so much faster?

It removed **recurrence**, the RNN's sequential hidden state, and replaced it with attention over all positions at once. Without a sequential dependency between positions, the whole sequence is processed in parallel during training, so GPUs are actually used rather than waiting on a chain of steps.

10. What fixes attention's blindness to order, and why fixed rather than learned encodings?

Position encodings. The original paper preferred fixed sinusoidal encodings because they require no training data and follow a regular pattern that extends to positions and sequence lengths never seen during training, whereas learned position embeddings exist only for the positions the training set happened to contain.

11. Self-attention vs cross-attention in the decoder; can they be swapped?

Decoder **self-attention** attends to the target tokens written so far, under a causal mask. **Cross-attention** attends to the encoder's representation of the source sentence, with no mask.

They cannot be swapped: they read different inputs under different masking. Without cross-attention the decoder never sees the source at all, so it cannot translate; without masked self-attention the output loses its own left context, and at inference it would be conditioning on tokens that have not been generated yet.

12. Layer norm: what it does, and why not batch norm?

Layer normalization keeps activations in a stable range so that deep stacks train quickly and without blowing up.

Batch norm normalizes across the batch, which breaks down exactly where language models live: variable-length sequences, small batches, and single-example inference, where batch statistics are unstable or meaningless. Layer norm normalizes within each token's own feature vector, so it behaves identically at training time and at inference time no matter the batch size.

13. What is the feed-forward network for, and why must it be non-linear?

Attention *gathers* information from other tokens; the FFN *processes* that information inside each token, position by position.

The two jobs are complementary, which is why every block has both.

The non-linearity is essential because stacked linear layers collapse mathematically into a single linear map. Without it, adding depth would add parameters but no new representable functions.

Part D. The agent exercise

What the mini-SWE-agent run was meant to show.

The whole agent is an LLM in a loop with a shell: no new machine learning anywhere in it. Each iteration the model proposes a command, the shell runs it, and the output is appended to the conversation, which is why the context grows every step.

The golden rules of AI coding that the run tends to demonstrate:

- **Verify with executed evidence.** Never accept "done" on the model's word; an agent that writes tests but never runs them has proved nothing.
- **Read every command before approving it.** The human in the loop is the safety boundary, and it only works if you actually read.
- **Work in a scratch directory, under version control.** Agents overwrite files cheerfully.
- **Give small tasks and check each step.** Errors compound quickly when each step's output becomes the next step's input.
- **Cap the budget.** Steps, cost and time limits live in the harness, not in the model: safety comes from the loop you build around it.

These are the same habits your Project 1 AI-use log is asking you to make explicit.